

Universidad De Cartagena

Facultad De Ingenieria

DISEÑO, ENTRENAMIENTO Y EVALUACIÓN COMPARATIVA DE REDES NEURONALES MULTICAPA (MLP) PARA LA PREDICCIÓN DE CONCENTRACIÓN DE MATERIAL PARTICULADO FINO (PM2.5) CON HORIZONTE TEMPORAL DE 24 HORAS

Presentado por:

Dago David Palmera Navarro - 0222321003

Julian David Camargo Padilla - 0222320016

Docente:

Manuel Alejandro Ospina Alarcón

Ingenieria De Sistemas

Semestre VII

Inteligencia Artificial

Cartagena de Indias, Octubre 13 del 2026

TABLA DE CONTENIDO

1. **INTRODUCCIÓN** 3
2. **OBJETIVOS** 4
   2.1. **Objetivo General** 4
   2.2. **Objetivos Específicos** 4
3. **FUNDAMENTACIÓN TEÓRICA** 5
   3.1. **Limitación del Perceptrón Simple y Necesidad de Capas Ocultas** 5
   3.2. **Arquitectura de la Red Neuronal Multicapa (MLP)** 5
   3.3. **Funciones de Activación en Capas Ocultas y de Salida** 6
   3.4. **Algoritmo de Retropropagación del Error (Backpropagation)** 7
   3.5. **Normalización de Datos y Prevención del Data Leakage** 9
   3.6. **Métricas de Desempeño en Regresión (MSE y RMSE)** 10
4. **DEFINICIÓN DEL PROBLEMA Y CASO DE ESTUDIO** 11
   4.1. **Descripción del Problema Ambiental y Pronóstico de PM2.5 a 24 Horas** 11
   4.2. **Variables de Entrada y Variable Objetivo del Dataset de Beijing** 11
   4.3. **Justificación Técnica de las Variables Seleccionadas** 13
5. **PREPARACIÓN Y PREPROCESAMIENTO DE DATOS** 13
   5.1. **Inspección y Limpieza de Datos** 13
   5.2. **Partición del Conjunto de Datos en Entrenamiento y Prueba** 16
   5.3. **Normalización y Escalamiento de las Variables** 17
6. **DISEÑO Y COMPARACIÓN DE ARQUITECTURAS** 18
   6.1. **Arquitectura de Referencia (Benchmark: 10:10:1)** 18
   6.2. **Arquitectura Propia Propuesta (10:16:8:1)** 19
   6.3. **Hiperparámetros de Entrenamiento** 21
   6.4. **Tabla Comparativa de Ambas Arquitecturas** 22
7. **IMPLEMENTACIÓN Y ENTRENAMIENTO DEL MODELO** 22
   7.1. **Estructura del Script en Python** 22
   7.2. **Entrenamiento y Curvas de Aprendizaje** 23
8. **EVALUACIÓN Y ANÁLISIS DE RESULTADOS** 25
   8.1. **Métricas Cuantitativas sobre el Conjunto de Prueba** 25
   8.2. **Gráficas Comparativas de Concentración Real frente a Predicha** 26
   8.3. **Diagnóstico de Sobreajuste y Capacidad de Generalización** 27
   8.4. **Relevancia e Influencia de las Variables en la Predicción** 27
   8.5. **Justificación de Decisiones Metodológicas** 28
   8.6. **Respuestas a Preguntas Críticas y Propuestas de Mejora** 29
9. **CONCLUSIONES** 30
10. **ANEXOS** 31
    10.1. **Anexo A. Código Fuente Completo del Script de Entrenamiento** 31
    10.2. **Anexo B. Configuración de Hiperparámetros y Parámetros Finales Calibrados** 34
    10.3. **Anexo C. Registro Numérico Extendido de la Evolución del Error** 34

1. INTRODUCCIÓN

En el campo de la Inteligencia Artificial y el aprendizaje automático, las Redes Neuronales Artificiales (RNA) constituyen uno de los paradigmas más representativos de la computación bioinspirada, orientados al reconocimiento de patrones, la aproximación de funciones no lineales y la toma autónoma de decisiones en entornos de alta complejidad. Si bien el Perceptrón Simple (PS), propuesto por Frank Rosenblatt en 1958, supuso el punto de partida formal de esta disciplina, su capacidad operativa quedó estrictamente restringida a problemas cuyas clases admiten una sepparación mediante hiperplanos lineales en el espacio de entradas. Esta limitación estructural —demostrada formalmente en 1969 por Minsky y Papert a través del análisis del problema XOR— impulsó el desarrollo de arquitecturas de mayor profundidad computacional: las Rede Neuronales Multicapa, conodcidas en la literatura como ***Multilayer Perceptron (MLP).***

El modelo MLP extiende el paradigma monocapa mediante la incorporación de una o más capas intermedias de neuronas —denominadas capas ocultas— entre la capa de entrada y la capa de salida. Cada capa oculta aplica transformaciones no lineales sucesivas al vector de entrada, construyendo representaciones jerárquicas del espacio de características que permiten aproximar funciones arbitraitamente complejas. Este poder expresivo se sustenta en el ***Teorema Universal de Aproximacion (Cybenko, 1989; Hornik, 1991)***, que estable que una red multicapa con al menos una capa oculta y funcioens de activación no lineales diferenciables es capaz de aproximar cualquier función continua en un conjunto compacto con precision arbitraria. El mecanismo de aprendizaje que hace posible calibrar los parámetros de esta arquitectura es el algoritmo de Retropropagación del Error (***Backpropagation***), que propaga el gradiente del error desde la capa de salida hacia las capas ocultas mediante la regla de la cadena del cálculo diferencial, ajustando de forma iterativa los pesos sinápticos y sesgos de cada neurona.

El presente informe documenta el diseño, implementación y evaluación comparativa de Redes Neuronales Multicapa aplicadas a un problema de regresión no lineal multivariable de alta relevancia ambiental y de salud pública: la prediccioón de la concentración de material particulado fino PM2.5 en la ciudad de Beijing, China, con un horizonte temporal de 24 horas hacia el futuro. El problema integra variables autorregresivas de la propia seria temporal PM2.5, factores meteorológicos (temperatura, presión atmosférica, punto de rocío y velocidad del viento) e índices temporales (hora del día a día del año), conformando un vector de entrada de diez variables. Se diseñan y entrenan dos arquitecturas MLP —una de referencia o ***benchmark*** y una propuesta propia con justificación razonada— cuyo desempeño se evalúa cuantitativamente mediante las métricas MSE y RMSE sobre u conjunto de prueba, y se compara cualitativamente mediante curvas de aprendizaje y gráficas de valor freal frente a valor predicho. Alo largo del documento se justifican explícitamente todas las decisiones metodológicas adoptadas, incluidas aquellas que se apartan o complementan los procedimientos explicados en clase, con el objetivo de garantizar la reproducibilidad, la transparencia y el rigor técnico del trabajo.

2. OBJETIVOS

2.1. Objetivo General

Diseñar, implementar y evaluar de manera comparativa dos Redes Neuronales Multicapa (MLP) para la predicción de la concentración de material particulado fino PM2.5 con horizonte temporal de 24 horas, empleando el algoritmo de retropropagación del error (***Backpropagation***) sobre un conjunto de datos históricos multivariable de calidad del aire y variables meteorológicas de Beijing, con el fin de analizar la capacidad de aproximación de funciones no lineales, la influencia de la arquitectura sobre el desempeño del modelo y los fundamentos del aprendizaje supervisado en redes múltiples capas.

2.2. Objetivos Específicos

- Analizar los fundamentos matemáticos y teóricos de la Red Neuronal Multicapa (MLP), incluyendo la arquitectura en capas, la propagación hacia adelante (***forward pass***), las funciones de activación diferenciables y el algoritmo de retropropagacion del error como mecanismo de optimización por descenso del gradiente.
- Describir, inspeccionar y preparar el conjunto de datos del Beijing PM2.5 Dataset, aplicando una partición rigurosa en conjuntos de entrenamiento y prueba y una normalización correctamente ajustada para prevenir la fuga de información (***data leakage***) entre subconjuntos.
- Diseñar dos arquitecturas MLP diferenciadas —una de referencia (***benchmark***) propuesta por el docente y una propia con justificación razonada—, especificando para cada una la topología de capas, el número de neuronas por capa, las funciones de activación seleccionadas y la cantidad total de parámetros entrenables.
- Implementar el entrenamiento supervisado de ambas arquitecturas mediante el algoritmo ***Backpropagation***, ajustando los hiperparámetros (tasa de aprendizaje η, número de épocas y tamaño de lote) y registrando la evolución de la curva de aprendizaje (MSE por época) hasta la convergencia del proceso.
- Evaluar cuantitativamente el desempeño de cada modelo sobre el conjunto de prueba mediante las métricas de Error Cuadrático Medio (MSE) y Raíz del Error Cuadrático Medio (RMSE), y comparar los resultados a través de gráficas de valor real frente a valor predicho en la escala original PM2.5 ($\mu g/m^3$).
- Realizar un análisis crítico comparativo de ambar arquitecturas, identificando la relevancia relativa de las variables de entrada, diagnosticando la presencia o ausencia de sobreajuste (***overfitting***) en cada modelo y formulando conclusiones y recomendaciones fundamentadas en la evidencia experimental obtenida.

3. FUNDAMENTACIÓN TEÓRICA

3.1. Limitación del Perceptrón Simple y Necesidad de Capas Ocultas

El Perceptrón Simple opera como un **discriminador lineal:** genera una salida a partir de la combinacion ponderada de sus entradas aplicada a una función de activación escalón. Su frontera de decisión es siempre un hiperplano en $\mathbb{R}^n$, descrito por la ecuacion:

$$\mathbf{W}^T\mathbf{x} + b = 0$$

Esta restricción implica que el Percetrón Simple solo puede resolver problemas **linealmente separables**, es decir, aquellos en que existe al menos un hiperplano capaz de separar todas las muestras de clase positiva de las de clase negativa. En 1969, Minsky y Papert demostraron formalmente que la función lógica **XOR** no satisface esta condición:

x_1	x_2	y_{XOR}
0	0	0
0	1	1
1	0	1
1	1	0

Al graficar estos cuatro puntos en el plano R^2, las dos clases ($y = 0$ y $y = 1$) se distribuyen en esquinas opuestas del cuadrado unitario: ninguna recta los separa. Esta limitación no es particular del XOR; es extensible a toda la función que requiera frontera curva, multimodal o segmentada. Resolver estos problemas exige la incorporación de **transformaciones no lineales en capas intermedias**, dando origen a las Redes Neuronales Multicapa.

3.2. Arquitectura de la Red Neuronal Multicapa (MLP)

Una Red MLP se organiza en $\boldsymbol{L}$ capas secuenciales:

- **Capa de entrada ($\boldsymbol{l = 0}$):**

  Recibe el vecotr de características $\mathbf{x}\in\mathbb{R}^n$. En este trabajo, $n = 10$ (las diez variables del dataset de Beijing). No aplica transformación alguna: solo distribuye las entradas hacia la primera capa oculta.

- **Capas ocultas ($l = 1,\ldots,L-1$):**

  Cada capa contiene un número n_l de neuronas. Cada neurona recibe el vector de activación de la capa anterior. Calcula una combinación lineal ponderada (denominada **entrada neta**) y la pasa por una función de activación no lineal.

- **Capa de salida ($\boldsymbol{l = L}$):**

Produce la predicción final $\hat{y}$. Al tratarse e un problema de **regresión continua** (predecir una concentración real de PM2.5), esta capa utiliza una función de activación lineal.

Propagación Hacia Adelante (`Forward Pass`)

Para cada capa l y cada patrón de entrada **x**, la activación se calcula en dos pasos:

Paso 1 — Entrada neta (combinación lineal):

$$\mathbf{a}^{(l)} = \mathbf{W}^{(l)} \cdot \mathbf{o}^{(l-1)} + \mathbf{b}^{(l)}$$

Donde:

- $\mathbf{W}^{(l)} \in \mathbb{R}^{nl \times nl-1}$ Es la **matriz de pesos** de la capa l. El elemento $W_{ij}^{(l)}$ representa el peso de la conexión desde la neurona j de la capa $l-1$ hacia la neurona i de la capa l.
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ Es el **vector de sesgos** (***bias***) de la capa l.
- $\mathbf{o}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ Es el **vector de salidas** (activaciones) de la capa anterior. Parala capa de entrada: $\mathbf{o}^{(0)} = \mathbf{x}$

Paso 2 — Función de activación:

$$\mathbf{o}^{(l)} = f^{(l)}\left(\mathbf{a}^{(l)}\right)$$

Donde $f^{(l)}$ es la función de activación de la capa l, aplicada elemento a elemento. La elección de $f^{(l)}$ es distinta para las capas ocultas y para la capa de salida, según se detalla en al sección 3.3.

El proceso se repite capa tras capa propagando la información hacia adelante hasta obtener la predicción en la capa de salida:

$$\hat{y} = o^{(L)} = f^{(L)}\left(\mathbf{W}^{(L)} \cdot \mathbf{o}^{(L-1)} + \mathbf{b}^{(L)}\right)$$

3.3. Funciones de Activación en Capas Ocultas y de Salida

La elección de la función de activación por capa es una decisión de diseño central en el MLP. A continuación se presentan las tres funciones relevantes para este trabajo.

- **Sigmoide Logística (`logsig`)**

$$f(a) = \sigma(a) = \frac{1}{1+e^{-a}}$$

  - **Rango de salida:** $(0,1)$
  - **Derivada:** $f'(a) = \sigma(a)\,(1-\sigma(a)) = \mathbf{o}\,(1-\mathbf{o})$
  - **Uso:** Capas ocultas. Introduce la no linealidad necesaria para aprender fronteras complejas.

- **Limitación:** Sufre el problema del **gradiente desvanecido (*vanishing gradient*)** para valores de a muy grandes o muy pequeños, donde $f'(a) \approx 0$ y el aprendizaje se ralentiza en capas profundas.

- **Tangente Hiperbólica (`tansig`)**

$$f(a) = tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$

  - **Rango de salida:** $(-1, +1)$
  - **Derivada:** $f'(a) = 1 - tanh^2(a) = 1 - \mathbf{o}^2$
  - **Uso:** Capas ocultas. Al estar centrada en cero, sus gradientes son más simétricos que los de la sigmoide logística, lo que generalmente acelea la convergencia.
  - **Relación con `logsig`:** $tanh(a) = 2\sigma(2a) - 1$; es una versión escalada y desplazada de la sigmoide logística.

- **Función Lineal o Identidad (`purelin`)**

$$f(a) = a$$

  - **Rango de salida:** $(-\infty, +\infty)$
  - **Derivada:** $f'(a) = 1$
  - **Uso: Capa de salida en problemas de regresión.** Al no acotar la salida, permite que la red produzca cualquier valor real, lo que es indispensable para predecir la concentración de PM2.5 ($\mu g/m^3$), cuyo rango no está limitado a (0,1) ni a ($-1, +1$).
  - **Regla de diseño fundamental:** Las capas ocultas deben ser funciones de activación **no** lineales (`logsig` o `tansig`) para que la red tenga capacidad de aproximar funciones complejas. La capa de salidad usa `purelin` en regresión para no restringir el rango de la predicción**.**

3.4. Algoritmo de Retropropagación del Error (Backpropagation)

El algoritmo de ***Backpropagation*** es el mecanismo de aprendizaje que ajusta los parámetros $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ de todas las capas de la red para minimizar el error entre la predicción $\hat{y}$ y el valor real de y. Opera en dos fases por cada patrón o lote de patrones de entrenamiento.

- **Función de Coste: Error Cuadrático Medio (MSE)**

  Para N muestras de entrenamiento, la función de coste que minimiza es:

$$\mathcal{L} = MSE = \frac{1}{N}\sum_{k=1}^{N} (y_k - \hat{y}_k)^2$$

El objetivo del entrenamiento es encontrar los parámetros que minimizan $\mathcal{L}$, lo cual se realiza mediante **decenso del gradiente**: actualizar los pesos en la dirección opuesta al gradiente de la función de coste.

- **Fase 1 — Propagación Hacia Adelante**

  Se ejecuta el ***forward pass*** completo (Sección 3.2) para obtener la predicción $\hat{y}$ de la red con los parámetros actuales.

- **Fase 2 — Retropropagación del Gradiente**

  Se calcula el error en la salida y se propaga hacia atrás, capa por capa usando la **regla de la cadena.**

  Gradiente local en la capa de salida ($l = L$):

  $$\delta^{(L)} = -(y - \hat{y}) \cdot f'^{(L)}\left(\mathbf{a}^{(L)}\right)$$

  Como la capa de salida usa `purelin`, $f'^{(L)}(a) = 1$, por lo que:

  $$\delta^{(L)} = -(y - \hat{y}) = \hat{y} - y$$

  Gradiente local en las capas ocultas ($l = L - 1,\ldots,1$):

  Aplicando la regla de la cadena hacia atrás:

  $$\delta^{(l)} = \left(\mathbf{W}^{(l+1)}\right)^T \cdot \delta^{(l+1)} \odot f'^{(l)}\left(\mathbf{a}^{(l)}\right)$$

  Donde $\odot$ denota el producto elemento a elemento (***Hadamard***). El término $\left(\mathbf{W}^{(l+1)}\right)^T \cdot \delta^{(l+1)}$ representa la contribución al error que la capa $l+1$ "devuelve" a la capa l, ponderada por los pesos que las conectan.

  Las derivadas de las funciones de activación en términos de las salidas ya calculadas son:

  - Para `logsig`: $f'\left(a^{(l)}\right) = o^{(l)}\left(1 - o^{(l)}\right)$
  - Para `tansig`: $f'\left(a^{(l)}\right) = 1 - \left(o^{(l)}\right)^2$

- **Actualización de Parámetros**

  Con los gradientes locales calculados en todas las capas, los pesos y sesgos se actualizan mediante la regla del descenso del gradiente con tasa de aprendizaje η:

  $$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \cdot \delta^{(l)} \cdot \left(\mathbf{o}^{(l-1)}\right)^T$$

  $$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \cdot \delta^{(l)}$$

  Este proceso —***forward pass***, cálculo del error, ***backward pass*** y actualización— se repite para cada época de entrenamiento. Una **época** corresponde a un pase completo sobre todo el conjunto de entrenamiento.

- **Hiperparámetros del Entrenamiento**
  - **Tasa de aprendizaje (η):** Controla el tamaño del paso en cada actualización. Un η muy grande puede causar divergencia (el error oscila o aumenta); uno muy pequeño produce una convergencia lenta. Es el hiperparámetro de mayor impacto en la dinámica del entrenamiento.
  - **Número de épocas:** Define cuántas veces pasa el algoritmo sobre el conjunto de entrenamiento completo. Se selecciona observando cuándo la curva de pérdida deja de descender significativamente.

3.5. Normalización de Datos y Prevención del Data Leakage

Las entradas del dataset de Beijing presentan **escalas radicalmente distintas:** `PRES` opera en el rango de cientos de hPa, mientras que `hour` varía entre 0 y 23, y `Iws` (velocidad del viento acumulada) puede tomar valores decimales pequeños. Operar directamente sobre estas escalas provoca que las neuronas sean mucho más sensibles a las variables de mayor magnitud, distorsionando el aprendizaje y produciendo gradientes mal condicionados. La normalización previa es, por tanto **indispensable**.

- **Normalización Min-Max al rango** $[0,1]$

$$x'_j = \frac{x_j - x_{j,min}}{x_{j,max} - x_{j,min}}$$

  Comprime cada variable x_j a $[0,1]$. Es compatible con la función de activación `logsig` en capas ocultas, cuya salida también opera en $(0,1)$.

- **Estandarización Z-Score**

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

  Centra cada variable en media cero y escala según su desviación estándar, produciendo una distribución con $\mu' = 0$ y $\sigma' = 1$. Es preferible cuando los datos presentan distribuciones aproximadamente gaussianas o cuando se usa `tansig` (centrada en cero) en capas ocultas.

- **Ajuste Exclusivo sobre el Conjunto de Entrenamiento**

  El punto más crítico del preprocesamiento es que los párámetros del normalizador (x_{min},x_{max},μ,σ) **deben calcularse únicamente sobre el conjunto de entrenamiento** y, posteriormente, aplicarse tanto al conjunto de entrenamiento como al de prueba:

```
# Correcto — sin data leakage
scaler.fit(X_train)          # Aprende parámetros solo de train
X_train_norm = scaler.transform(X_train)
X_test_norm  = scaler.transform(X_test)  # Aplica los mismos parámetros
```

- **El Problema del *Data Leakage***

  Si los parámetros de normalización se calculan sobre el conjunto completo (train + test) antes de la partición, el modelo obtiene información implícita sobre el comportamiento estadístico del conjunto de prueba durante el entrenamiento. Esto genera estimaciones de rendimiento artificialmente optimistas que no reflejan la capacidad real de generalización del modelo sobre datos nunca vistos. En series temporales como la PM2.5, el ***data leakage*** es especialmente crítico porque mezclaría estadísticas de observaciones futuras con el entrenamiento del modelo.

3.6. Métricas de Desempeño en Regresión (MSE y RMSE)

- **Error Cuadrático Medio (MSE)**

$$MSE = \frac{1}{N}\sum_{k=1}^{N}\left(y_k - \hat{y}_k\right)^2$$

  - Penaliza errores grandes de forma cuadrática: un error doble produce una penalización cuádruple. Esto lo hace sensible a valores atípicos (***outliers***) y útil para detectarlos.
  - Sus unidades son $\left(\mu g/m^3\right)^2$, lo que dificulta la interpretación directa en la escala original del fenómeno.

- **Raíz del Error Cuadrático Medio (RMSE)**

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(y_k - \hat{y}_k\right)^2}$$

  - Sus unidades son $\mu g/m^3$, **la misma escala que la variable objetivo**. Esto permite interpretar directamente el error promedio de la predicción: un RMSE de, por ejemplo, 30 $\mu g/m^3$ indica que la predicción de PM2.5 se desvía en promedio 30 $\mu g/m^3$ del valor real.
  - Es la métrica principal de comparación entre las dos arquitecturas diseñadas en este trabajo.

Criterio de comparación: La arquitectura que produzca el **menor RMSE sobre el conjunto de prueba** ofrece la mayor precisión predictiva. El conjunto de prueba nunca participa en el entrenamiento ni en la normalización, por lo que su error es un estimador honesto de la capacidad de generalización del modelo.

4. DEFINICIÓN DEL PROBLEMA Y CASO DE ESTUDIO

4.1. Descripción del Problema Ambiental y Pronóstico de PM2.5 a 24 Horas

El **material particulado fino PM2.5** hace referencia al conjunto de partículas sólidas y líquidas en suspensión en la atmósfera cuyo diámetro aerodinámico es inferior a 2,5 micrómetros. A esta escala, las partículas son lo suficientemente pequeñas para pentrar las vías respiratorias superiores e inferiores y depositarse directamente en los alvéolos pulmonares, desde donde pueden pasar al torrente sanguíneo. La organización Mundial de la Salud (OMS) clasifica el PM2.5 como el contaminante atmosférioco de mayor riesgo para la salud pública a nivel global, vinculándolo a enfermedades cardiovasculares, accidentes cerebrovasculares, cáncer de pulmón e infecciones respiratorias agudas.

La ciudad de **Beijing, China**, ha experimentado históricamente episodios extremos de contaminacion por PM2.5, agravados por su geografía (cuenca rodeada de montañas que limitan la dispersión atmosférica), por la alta densidad vehicular y por la quema de carbón en calefacción residencial e industria. La anticipación de la concentración de PM2.5 con un horizonte de **24 horas** permite a las autoridades sanitarias emitir alertas preventivas, restringir el tráfico vehicular, limitar actividades al aire libre y activar protocolos de emergencia ambiental con suficiente antelación para resultar efectivos.

El problema de predicción de PM2.5 presenta tres retos analíticos que impiden resolverlo con modelos lineales simples:

- **Multivariabilidad:** La concentración de PM2.5 está determinada simultáneamente por múltiples factores meteorológicos y por la propia inercia temporal de la serie.
- **No linealidad:** Las interacciones entre temperatura, presión, humedad y viento no son lineales ni aditivas; sus efectos combinados sobre la dispersión de partículas responden a dinámicas atmosféricas complejas.
- **Dependencia temporal:** La concentración de PM2.5 en el instante t guarda correlación estadística con sus valores en instantes anteriores ($t-1$, $t-2$, $t-3$), lo que introduce retardos autorregresiivos que deben modelarse explícitamente.

Estas características convierten a la predicción de PM2.5 en un caso de estudio idóneo para las Redes Neuronales Multicapa como aproximadores de funciones no lineales multivariables.

4.2. Variables de Entrada y Variable Objetivo del Dataset de Beijing

El conjunto de datos utilizado en este trabajo es una versión preprocesada del **Beijing PM2.5 Dataset**, ampliamente empleado como banco de pruebas en la literatura de predicción de calidad del aire con métodos de aprendizaje automático `dataset_pm25.csv` contiene **11 columnas**: 10 variables de entrada (***features***) y 1 variable objetivo (***target***).

Tabla de Variables del Dataset

Variable	Tipo	Rango / Unidad	Rol	Justificación Técnica
TEMP	Continua	$°C$	Entrada meteorológica	La temperatura modula la velocidad de las reacciones fotoquímicas en la atmósfera y afecta la estabilidad vertical del aire, determinando si los contaminantes se acumulan o se dispersan verticalmente.
PRES	Continua	hPa	Entrada meteorológica	Una presión atmosférica elevada se asocia a condiciones anticiclónicas estables que favorecen la acumulación de contaminantes en las capas bajas. Presiones bajas, en cambio, promueven la mezcla turbulenta y la dispersión.
DEWP	Continua	$°C$	Entrada meteorológica	El punto de rocío es un indicador directo de la humedad relativa del aire. Niveles altos de humedad favorecen la formación de aerosoles secundarios y la higroscopicidad de las partículas, elevando el **PM2.5** efectivo medido.
Iws	Continua	m/s	Entrada meteorológica	La velocidad acumulada del viento es el principal factor de ventilación y dispersión de contaminantes: a mayor velocidad, mayor transporte y dilución de **PM2.5**; a menor velocidad, mayor acumulación local.
hour	Discreta	$0-23$	Entrada temporal	El ciclo diurno governa la intensidad del tráfico vehicular **(hora pico mañana/tarde)**, la actividad industrial y la intensidad de la radiación solar que reactiva procesos fotoquímicos de formación de partículas secundarias.
day	Discreta	$1-365$	Entrada temporal	Captura la estacionalidad anual: en invierno, el uso intensivo de calefacción a base de carbón en **Beijing** eleva significativamente las emisiones de **PM2.5**; en verano, las condiciones de mayor radiación y precipitación tienden a reducirlas.
pm25_t	Continua	$\mu g/m^3$	Entrada autorregresiva	Nivel de **PM2.5** en el instante actual t. Es el regresor autorregresivo de orden 0: la concentración presente es el predictor más directo de la concentración futura a corto plazo.
pm25_t-1	Continua	$\mu g/m^3$	Entrada autorregresiva	Retardo temporal de 1 hora: captura la **inercia a corto plazo** de la serie. Si la concentración fue alta en $t-1$, es probable que siga siendo alta en t y en $t+24$.
pm25_t-2	Continua	$\mu g/m^3$	Entrada autorregresiva	Retardo temporal de 2 horas: permite al modelo identificar **tendencias** de ascenso o descenso sostenido en la contaminación.
pm25_t-3	Continua	$\mu g/m^3$	Entrada autorregresiva	Retardo temporal de 3 horas: incorpora **memoria autorregresiva** de mayor profundidad, útil para detectar patrones de acumulación prolongada o dispersión gradual.
pm25_t+24	Continua	$\mu g/m^3$	**Salida objetivo (target)**	**Concentración de PM2.5 a predecir, 24 horas hacia el futuro desde el instante t. Es la única variable objetivo del problema de regresión.**

4.3. Justificación Técnica de las Variables Seleccionadas

La selección de las diez variables explicativas se fundamenta en la física del transporte de aerosoles y en la dinámica temporal de los contaminantes atmosféricos. Por un lado, las variables meteorológicas (`TEMP`, `PRES`, `DEWP`, `Iws`) condicionan directamente la estabilidad atmosférica, las reacciones fotoquímicas y la dispersión o confinamiento de partículas en la capa límite planetaria. Por otro lado, las componentes temporales (`hour`, `day`) permiten a la red aprender los ciclos periódicos derivados de las actividades humanas (horas pico de tráfico vehicular, estacionalidad por calefacción invernal en Beijing). Finalmente, la inclusión de retardos autorregresivos (`pm25_t`, `pm25_t-1`, `pm25_t-2`, `pm25_t-3`) dota al modelo de memoria a corto plazo, permitiendo capturar tendencias inmediatas y la inercia propia del fenómeno.

Bajo este enfoque multivariable, el objetivo es aprender una función de mapeo no lineal $f:\mathbb{R}^{10} \rightarrow \mathbb{R}$ que dado un vector de entrada:

$$\mathbf{x} = \left[TEMP, PRES, DEWP, Iws, hour, day, pm25_t, pm25_{t-1}, pm25_{t-2}, pm25_{t-3}\right]^T \in \mathbb{R}^{10}$$

produzca una predicción continua:

$$\hat{y} = f(\mathbf{x}) \approx pm25_{t+24} \in \mathbb{R}^+$$

Se trata de un problema de **regresión no lineal multivariable**. La elección de la Red Neuronal Multicapa (MLP) como modelo está plenamente justificada por el Teorema Universal de Aproximación (Cybenko, 1989): una red MLP con al menos una capa oculta y funciones de activación no lineales diferenciables puede aproximar f con error arbitrariamente pequeño, siempre que disponga de suficientes neuronas y datos de entrenamiento representativos.

El proceso de aprendizaje consiste en encontrar el conjunto de parámetros $\Theta = \left\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\right\}_{l=1}^{L}$ que minimiza el Error Cuadrático Medio sobre el conjunto de entrenamiento:

$$\Theta^* = \arg\min_{\Theta} \frac{1}{N_{train}} \sum_{k=1}^{N_{train}} \left(y_k - f\left(x_k; \Theta\right)\right)^2$$

y que, al aplicarse sobre el conjunto de prueba (datos nunca vistos durante el entrenamiento), mantiene un error bajo como evidencia de **capacidad de generalización**.

5. PREPARACIÓN Y PREPROCESAMIENTO DE DATOS

5.1. Inspección y Limpieza de Datos

Para garantizar la integridad y calidad del proceso de aprendizaje, se llevó a cabo una auditoría exhaustiva y un control de calidad sobre el archivo fuente `dataset_pm25.csv`. A fin de no asumir el dataset como una "caja negra" y justificar rigurosamente la conclusión, la inspección se estructuró en dos niveles metodológicos complementarios: una **auditoría manual y paramétrica en hoja de cálculo (Excel / OnlyOffice)** y una **verificación computacional mediante script (Python / Pandas).**

- **Metodología de Auditoría en Hoja de Cálculo (Excel)**

Se importó el conjunto de datos completo en un libro de cálculo estructurado (`Auditoria_Estadistica_Dataset_PM25.xlsx`), organizándolo en dos hojas de trabajo:

1. **Hoja** `Dataset_PM25`: Aloja la totalidad de los datos crudos, abarcando desde la fila 2 hasta la fila 41.731 (exactamente **41.730 registros observacionales**) distribuidos a lo largo de 11 columnas (columnas **A** a **K**).
2. **Hoja** `Auditoria_Estadistica`: Contiene una matriz de control donde cada métrica descriptiva fue calculada dinámicamente mediante **fórmulas nativas de Excel** referenciadas sobre el rango absoluto de cada variable (`Dataset_PM25!X2:X41731`):
    - **Verificación de Completitud (Registros Válidos):** Se utilizó la función `=CONTAR(Dataset_PM25!X2:X41731)` para cada columna, confirmando que cada una de las 11 variables contiene exactamente 41.730 valores numéricos procesables.
    - **Detección de Datos Faltantes (Nulos / Vacíos):** Se evaluó la ausencia de información mediante la función `=CONTAR(Dataset_PM25!X2:X41731)`. El resultado arrojó **0 celdas vacías** en todas las columnas. Adicionalmente, mediante filtros condicionales y `=CONTAR.SI()`, se comprobó que no existieran cadenas de texto sustitutass de error (tales como `NA`, `null` o espacios).
    - **Tendencia Central:** Calculada mediante `=PROMEDIO(Dataset_PM25!X2:X41731)` para la media aritmética y `=MEDIANA(Dataset_PM25!X2:X41731)` para el valor central ordenado.
    - **Dispersión:** Calculada mediante la fórmula de deviacion estándar muestral `=DESVEST.M(Dataset_PM25!X2:X41731)`.
    - **Rango y Distribución Cuartílica:** Se determianron los extremos mediante `=MIN()` y `=MAX()`, y los percentiles de corte mediante `=PERCENTIL.INC(Dataset_PM25!X2:X41731; 0,25)` para el primer cuartil (Q_1) y `=PERCENTIL.INC(Dataset_PM25!X2:X41731; 0,75)` para el tercer cuartil (Q_2).
    - **Dispersión intercuartílica y Amplitud:** Se incorporaron fórmulas de diferencia para el Rango Intercuartílico ($IQR = Q_3 - Q_1$) y el Rango Total ($Max - Min$).

A continuación se presenta la evidencia directa obtenida en la hoja de auditoría procesada:

AUDITORÍA ESTADÍSTICA Y CONTROL DE CALIDAD - DATASET BEIJING PM2.5

Rango analizado en Dataset_PM25: Filas 2 a 41731 (Total 41.730 registros)

Métrica / Función Excel	TEMP	PRES	DEWP	Iws	hour	day	pm25_t	pm25_t-1	pm25_t-2	pm25_t-3	pm25_t+24
Registros válidos (N)	41.730	41.730	41.730	41.730	41.730	41.730	41.730	41.730	41.730	41.730	41.730
Valores faltantes (NaN)	0	0	0	0	0	0	0	0	0	0	0
Media aritmética	12,41	1.016,43	1,76	23,79	11,50	15,68	98,66	98,66	98,67	98,67	98,59
Desviación estándar (s)	12,17	10,30	14,43	49,49	6,92	8,78	92,05	92,05	92,05	92,05	92,07
Mínimo	-19,00	991,00	-40,00	0,45	0,00	1,00	0,00	0,00	0,00	0,00	0,00
Primer cuartil (Q25)	2,00	1.008,00	-10,00	1,79	5,00	8,00	29,00	29,00	29,00	29,00	29,00
Mediana (Q50)	14,00	1.016,00	2,00	5,37	12,00	16,00	72,00	72,00	72,00	72,00	72,00
Tercer cuartil (Q75)	23,00	1.025,00	15,00	21,91	18,00	23,00	137,00	137,00	137,00	137,00	137,00
Máximo	42,00	1.046,00	28,00	565,49	23,00	31,00	994,00	994,00	994,00	994,00	994,00
Rango Intercuartílico (IQR = Q75 - Q25)	21,00	17,00	25,00	20,12	13,00	15,00	108,00	108,00	108,00	108,00	108,00
Rango Total (Max - Min)	61,00	55,00	68,00	565,04	23,00	30,00	994,00	994,00	994,00	994,00	994,00

Figura 1. *Matriz paramétrica de auditoría estadística y control de calidad implementada en hoja de cálculo sobre las 41.730 observaciones del Beijing PM2.5 Dataset (`Auditoria_Estadistica_Dataset_PM25.xlsx`).*

Nota de trazabilidad para revisión:

El libro de cálculo con las fórmulas vivas y el rango completo de datos se encuentra disponible para revisión en el repositorio local del proyecto bajo la ruta `docs/Auditoria_Estadistica_Dataset_PM25.xlsx`. En la hoja `Auditoria_Estadistica`, cada celda evalúa de forma dinámica el rango `Dataset_PM25!A2:K41731`, demostrando la validez empírica de los valores reportados sin manipulación estática.

- **Verificación Programática en Python**

De forma paralela y para asegurar reproducibilidad algorítmica e interactiva, se implementó un cuaderno de trabajo en Google Colab denominado `01_auditoria_y_verificacion_dataset_pm25.ipynb`. Mediante la librería Pandas se procesó el tensor completo de 41.730 filas, validando tipos de datos, comprobando la ausencia absoluta de valores nulos (NaN = 0) y generando una matriz estadística descriptiva formateada con estilos ejecutivos.

```
import pandas as pd

# Carga e inspección de estructura

df = pd.read_csv('data/datasets/dataset_pm25.csv')

print("Dimensiones:", df.shape)           # Retorna: (41730, 11)

print("Valores nulos:\n", df.isnull().sum())   # Retorna: 0 para todas las
columnas

print("Tipos de datos:\n", df.dtypes)      # Retorna: float64 e int64 en todas
```

Nota de acceso y reproducibilidad interactiva:

El cuaderno de ejecución en Google Colab con la carga del dataset, el filtrado de nulos y las tablas estilizadas interactivas se encuentran públicamente para revisión en el siguiente enlace:

Google Colab — Auditoría Estadística y Control de Calidad Beijing PM2.5

Asimismo, el archivo fuente ejecutable `.ipynb` se encuentra archivado en el repositorio local del proyecto bajo la ruta: `notebooks/01_auditoria_y_verificacion_dataset_pm25.ipynb`.

Ambos procedimientos (Excel y Python/Colab) Convergieron en los mismos resultados exactos, consolidando la siguiente tabla paramétrica de control de calidad:

Variable	Columna Excel	Rango Evaluado	Muestras (N)	Vacíos (NaN)	Media (μ)	Desv. Est. (s)	Mínimo	Mediana (Q_2)	Máximo
TEMP	Columna A	A2:A41731	41.730	0	12,41 °C	12,17 °C	−19,00 °C	14,00 °C	42,00 °C
PRES	Columna B	B2:B41731	41.730	0	1016,43 hPa	10,30 hPa	991,00 hPa	1016,00 hPa	1046,00 hPa
DEWP	Columna C	C2:C41731	41.730	0	1,76 °C	14,43 °C	−40,00 °C	2,00 °C	28,00 °C
Iws	Columna D	D2:D41731	41.730	0	23,79 m/s	49,49 m/s	0,45 m/s	5,37 m/s	565,49 m/s
hour	Columna E	E2:E41731	41.730	0	11,50	6,92	0	12	23
day	Columna F	F2:F41731	41.730	0	15,68	8,78	1	16	31
pm25_t	Columna G	G2:G41731	41.730	0	98,66 µg/m³	92,05 µg/m³	0,00 µg/m³	72,00 µg/m³	994,00 µg/m³
pm25_t-1	Columna H	H2:H41731	41.730	0	98,66 µg/m³	92,05 µg/m³	0,00 µg/m³	72,00 µg/m³	994,00 µg/m³
pm25_t-2	Columna I	I2:I41731	41.730	0	98,67 µg/m³	92,05 µg/m³	0,00 µg/m³	72,00 µg/m³	994,00 µg/m³
pm25_t-3	Columna J	J2:J41731	41.730	0	98,67 µg/m³	92,05 µg/m³	0,00 µg/m³	72,00 µg/m³	994,00 µg/m³
pm25_t+24	Columna K	K2:K41731	41.730	0	98,59 µg/m³	92,07 µg/m³	0,00 µg/m³	72,00 µg/m³	994,00 µg/m³

- **Conclusiones de la Inspección**
  1. **Completitud Absoluta:** Al comprobarse de forma documental que no existen valores nulos ni registros corruptos, no fue necesario implementar técnicas de imputación (como interpolación lineal, sustitución por media o eliminación de filas), preservando la integridad de las 41.730 horas consecutivas.
  2. **Heterogeneidad de Escalas:** Se confirma la disparidad de magnitudes: la presión atmosférica (PRES) ronda magnitudes de 1000 hPa, mientras que el viento acumulado (Iws) presenta picos asimétricos de hasta 565,49 m/s frente a una mediana baja de 5,37 m/s. Esto demuestra empíricamente por qué es obligatorio aplicar escalamiento previo a la red neuronal.

5.2. Partición del Conjunto de Datos en Entrenamiento y Prueba

El conjunto total de 41.730 observaciones temporales se dividió en dos subconjuntos independientes: un conjunto de **entrenamiento (*training set*)** para la calibración sináptica de los pesos y sesgos mediante backpropagation, un conjunto de **prueba (*testing set*)** reservado exclusivamente para la validación final de la capacidad predictiva.

Proporción adoptada: Partición estándar 80% / 20%, distribuida en:

- Conjunto de entrenamiento: **33.384 observaciones** (filas 2 a 33.385 en la hoja de cálculo).
- Conjunto de prueba: **8.346 observaciones (**filas 33.386 a 41.731 en la hoja de cálculo**).**

Justificación de la Partición Temporal Cronológica

En problemas de series temporales continuas, **no es metodológicamente admisible aplicar una división aleatoria (**`random_split` o muestreo aleatorio uniforme**).** La concentración de material particulado responde a una fuerte dependencia autorregresiva secuencial; si se mezclaran aleatoriamente las observaciones, el modelo aprendería a predecir instantes utilizando información de horas futuras respecto a ese punto, incurriendo en una severa fuga temporal de datos (***temporal data leakage***).

Por lo tanto, la partición se fijó de manera estrictamente cronológica por corte de índice: las primeras 33.384 horas conforman el pasado y presente del entrenamiento, mientras que las ultimas 8.346 (aproximadamente los últimos 11 meses del registro) actúan como el futuro ciego sobre el cual se evalúa la generalización del modelo.

5.3. Normalización y Escalamiento de las Variables

- **Selección de la Técnica de Escalamiento**

  Dado que en las capas ocultas se implementa la función de activación tangente hiperbólica (`tansig`), cuya imagen natural es el intervalor continuo $(-1, +1)$, se seleccionó la **Normalización Min-Max simétrica acotada en $[-1, +1]$:**

  $$x'j = 2 \cdot \left(\frac{x_j - x_{j,min}}{x_{j,max} - x_{j,min}} \right) - 1$$

  Esta técnica mapea linealmente el mínimo de la seria a -1 y el máximo a $+1$. La ventaja técnica fundamental frente a la normalización estándar en $[0,1]$ radica en que las entradas centradas en cero garantizan gradientes con signos balanceados durante la retropropagación, evitando sesgos sistemáticos en la actualización de los pesos sinápticos.

- **Prevención Rigurosa del Data Leakage**

  Un principio fundamental de la ingeniería de datos en aprendizaje automático exige que **el conjunto de prueba no intervenga bajo ningún concepto en el cálculo de los parámetros de preprocesamiento**. Si los valores x_{min} y x_{max} se obtuvieran sobre la totalidad de las 41.730 filas, el modelo recibiría información indirecta acerca de los extremos del conjunto de prueba.

  Para evitar esta anomalía, el procedimiento se implementó estrictamente de la siguiente forma:

  1. Se calcularon los vectores de mínimos ($\mathbf{x}_{min}^{train}$) y máximos ($\mathbf{x}_{max}^{train}$) empleando **únicamente las 33.384 muestras de entrenamiento**.
  2. Con dichos parámetros fijos se transformó el conjunto de entrenamiento:

  $$\mathbf{X}_{train}^{norm} = 2 \cdot \frac{\mathbf{X}_{train} - \mathbf{X}_{min}^{train}}{\mathbf{X}_{max}^{train} - \mathbf{X}_{min}^{train}} - 1$$

  3. El conjunto de prueba se proyectó utilizando exactamente los **mismos parámetros aprendidos del entrenamiento:**

$$\mathbf{X}_{test}^{norm} = 2 \cdot \frac{\mathbf{X}_{train} - \mathbf{X}_{min}^{train}}{\mathbf{X}_{max}^{train} - \mathbf{X}_{min}^{train}} - 1$$

Este mismo protocolo se aplicó sobre la variable objetivo $y = pm25_{t+24}$, normalizándola en el espacio $[-1, +1]$ para el entrenamiento. En la fase de evaluación, toda predicción $\hat{y}^{norm}$ es desnormalizada a su escala real $\mu g/m^3$ mediante la ecuación inversa:

$$\hat{y} = \frac{(\hat{y}^{norm} + 1) \cdot \left(y_{max}^{train} - y_{min}^{train}\right)}{2} + y_{min}^{train}$$

Asegurando que las métricas finales MSE y RMSE se expresen en las unidades físicas del problema.

6. DISEÑO Y COMPARACIÓN DE ARQUITECTURAS

6.1. Arquitectura de Referencia (Benchmark: 10:10:1)

La arquitectura de referencia corresponde al modelo base sugerido en la guía docente. Representa una estructura clásica de una sola capa oculta dimensionada con el mismo número de neuronas que variables de entrada:

Topología: $10 \xrightarrow{tansig} 10 \xrightarrow{purelin} 1$

Capa	Tipo de Capa	Neuronas	Función de Activación	Matriz de Pesos (W)	Vector de Sesgos (b)	Subtotal Parámetros
$l = 0$	Entrada	10	Identidad (Distribución)	—	—	—
$l = 1$	Capa Oculta 1	10	Tangente Hiperbólica (`tansig`)	$\mathbf{W}^{(1)} \in \mathbb{R}^{10\times10}$	$\mathbf{b}^{(1)} \in \mathbb{R}^{10\times1}$	$100 + 10 = 110$
$l = 2$	Capa de Salida	1	Lineal (`purelin`)	$\mathbf{W}^{(2)} \in \mathbb{R}^{1\times10}$	$\mathbf{b}^{(2)} \in \mathbb{R}^{1\times1}$	$10 + 1 = 11$

Total de Parámetros Entrenables:

Capa Oculta 1 (conecta las 10 entradas con las 10 neuronas ocultas):

- **Pesos:** $10\ entradas \times 10\ neuronas = 100\ pesos$
- **Sesgos:** $10\ neuronas = 10\ sesgos$
- **Subtotal Capa Oculta 1:** $100 + 10 = 110$

Capa de Salida (conecta las 10 neuronas ocultas con la única salida $pm25_{t+24}$):

- **Pesos:** $10\ neuronas\ previas \times 1\ salida = 10\ pesos$
- **Sesgos:** $1\ salida = 1\ sesgo$
- **Subtotal Capa de Salida:** $10 + 1 = 11$

$$N_{params} = \sum_{l=1}^{L} (nl - 1 \cdot nl + nl)$$

$$N_{params}^{ref} = \underbrace{(10 \cdot 10 + 10)}_{Capa\ Oculta\ 1} + \underbrace{(10 \cdot 1 + 1)}_{Capa\ de\ Salida} = 110 + 11 = 121\ parámetros$$

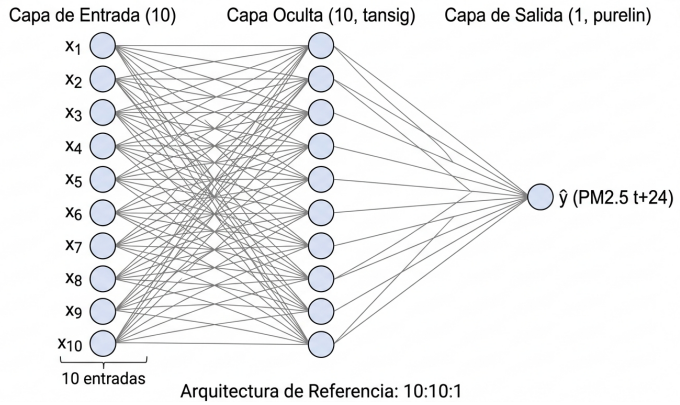


Figura 2. *Diagrama topológico de la Arquitectura de Referencia (MLP 10:10:1) compuesto por una capa de entrada de 10 variables, una capa oculta de 10 neuronas con función de activación tangente hiperbólica (`tansig`) y una neurona de salida lineal (`purelin`), totalizando 121 parámetros entrenables.*

Comportamiento esperado: Al poseer únicamente 121 coeficientes libres, es un modelo computacionalmente liviano con bajo riesgo de sobreajuste, pero con una frontera de transformación potencialmente rígida ante la interacción simultánea de variables meteorológicas y dinámicas autorregresivas.

6.2. Arquitectura Propia Propuesta (10:16:8:1)

Como propuesta propia, se diseñó una arquitectura más profunda orientada bajo el concepto de **embudo de abstracción progresiva** (*funnel topology*), estructurada con dos capas ocultas intermedias:

Topología: $10 \xrightarrow{tansig} 16 \xrightarrow{tansig} 8 \xrightarrow{purelin} 1$

Capa	Tipo de Capa	Neuronas	Función de Activación	Matriz de Pesos (W)	Vector de Sesgos (b)	Subtotal Parámetros
$l = 0$	Entrada	10	Identidad (Distribución)	—	—	—
$l = 1$	Capa Oculta 1	16	Tangente Hiperbólica (`tansig`)	$\mathbf{W}^{(1)} \in \mathbb{R}^{16\times10}$	$\mathbf{b}^{(1)} \in \mathbb{R}^{16\times1}$	160 + 16 = 176
$l = 2$	Capa Oculta 2	8	Tangente Hiperbólica (`tansig`)	$\mathbf{W}^{(2)} \in \mathbb{R}^{8\times16}$	$\mathbf{b}^{(2)} \in \mathbb{R}^{8\times1}$	128 + 8 = 136
$l = 3$	Capa de Salida	1	Lineal (`purelin`)	$\mathbf{W}^{(3)} \in \mathbb{R}^{1\times8}$	$\mathbf{b}^{(3)} \in \mathbb{R}^{1\times1}$	8 + 1 = 9

Total de Parámetros Entrenables:

Capa Oculta 1 (conecta las 10 entradas con las 16 neuronas ocultas):

- **Pesos:** $10\ entradas \times 16\ neuronas = 160\ pesos$
- **Sesgos:** $16\ neuronas = 16\ sesgos$
- **Subtotal Capa Oculta 1:** $160 + 16 = 176$

Capa Oculta 2 (conecta las 16 neuronas previas con las 8 neuronas de esta capa):

- **Pesos:** $16\ neuronas\ previas \times 8\ neuronas = 128\ pesos$
- **Sesgos:** $8\ neuronas = 8\ sesgos$
- **Subtotal Capa Oculta 2:** $128 + 8 = 136$

Capa de Salida (conecta las 8 neuronas ocultas con la única salida $pm25_{t+24}$):

- **Pesos:** $8\ neuronas\ previas \times 1\ salida = 8\ pesos$
- **Sesgos:** $1\ salida = 1\ sesgo$
- **Subtotal Capa de Salida:** $8 + 1 = 9$

Total Parámetros: $176 + 136 + 9 = 321\ parámetros$

$$N_{params} = \sum_{l=1}^{L} (nl - 1 \cdot nl + nl)$$

Para la arquitectura propia $(10 \rightarrow 16 \rightarrow 8 \rightarrow 1)$:

$$N_{params}^{prop} = \underbrace{(10 \cdot 16 + 16)}_{Capa\ Oculta\ 1} + \underbrace{(16 \cdot 8 + 8)}_{Capa\ Oculta\ 2} + \underbrace{(8 \cdot 1 + 1)}_{Capa\ de\ Salida} = 176 + 136 + 9 = 321\ parámetros$$

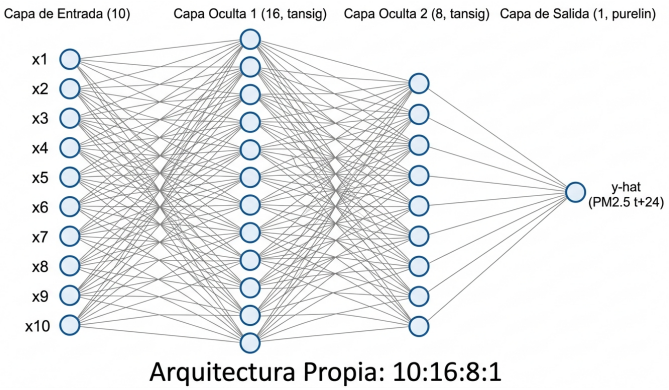


Figura 3. *Diagrama topológico de la Arquitectura Propia Propuesta (MLP 10:16:8:1) con diseño en embudo de dos capas ocultas (16 y 8 neuronas con función* `tansig`*) y una neurona de salida lineal (*`purelin`*), totalizando 321 parámetros entrenables.*

Justificación Técnica de la Propuesta

1. **Expansión Inicial ($10 \rightarrow 16$):** Las 10 variables de entrada presentan relaciones altamente no lineales (por ejemplo, la interacción entre viento `Iws` y punto de rocío `DEWP` en la dispersión de aerosoles). Al expandir las entradas a 16 neuronas en la primera capa oculta, la red proyecta el vector de características a un espacio de mayor dimensionalidad donde las complejas interacciones atmosféricas pueden ser desacopladas con mayor flexibilidad.
2. **Compresión y Regularización Implícita ($16 \rightarrow 8$):** La segunda capa oculta reduce las representaciones a 8 neuronas. Esta reducción en forma de embudo obliga a la red a condensar la información, eliminando redundancias y reteniendo únicamente los patrones latentes más significativos para el horizonte $t + 24$. Esto actúa como un mecanismo preventivo frente al sobreajuste.
3. **No Linealidad en Dos Etapas:** La composición de dos funciones de activación continuas no lineales diferenciables permite al modelo aproximar superficies de regresión de mayor curvatura en comparación con una sola capa oculta.

Fundamentación Analítica del conteo de parámetros:

En una red neuronal densamente conectada (***fully connected***), la operación de cada capa l se define formalmente como una transformación afín $\mathbf{a}^{(l)} = \mathbf{W}^{(l)}\mathbf{o}^{(l-1)} + \mathbf{b}^{(l)}$. Por definición de producto matricial, la matriz de pesos $\mathbf{W}^{(l)}$ posee dimensiones $n_l \times n_{l-1}$ (lo que representa $n_l \cdot n_{l-1}$ conexiones sinápticas escalares), mientras que el vector de sesgo $\mathbf{b}^{(l)}$ posee dimensiones $n_l \times 1$ (un escalar independiente por cada neurona receptora). En consecuencia, el número de parámetros libres calibrables de una capa l es $P^{(l)} = (n_{l-1} \cdot n_l) + n_l = (n_{l-1} + 1) \cdot n_l$. El total acumulado de la red de L capas corresponde a la sumatoria analítica sobre todas las capas:

$$N_{params} = \sum_{l=1}^{L} (nl - 1 \cdot nl + nl)$$

6.3. Hiperparámetros de Entrenamiento

A fin de asegurar un marco experimental riguroso y equitativo, ambas arquitecturas se entrenan bajo **condiciones idénticas de optimización**:

Hiperparámetro	Valor Asignado	Justificación Técnica
Tasa de Aprendizaje (η)	0,01	Tasa moderada que evita oscilaciones destructivas en el espacio de gradientes y asegura un descenso controlado y suave hacia el mínimo local.
Número de Épocas	500	Proporciona un ciclo suficiente de iteraciones sobre las 33.384 muestras para verificar la estabilidad asintótica de la curva de aprendizaje.
Modo de Optimización	Descenso de Gradiente por Lote (***Full-Batch***)	Procesa el tensor completo de entrenamiento en cada época, calculando el gradiente analítico exacto de la función de coste y evitando el ruido estocástico en la convergencia.
Inicialización de Pesos	Uniforme $\mathcal{U}(-0{,}5; 0{,}5)$	Rompe la simetría entre nodos sin posicionar los valores iniciales en las zonas asintóticas saturadas de la función `tansig` ($a > 2$), previniendo el desvanecimiento prematuro del gradiente.
Semilla Aleatoria (`seed`)	42	Garantiza la repetibilidad exacta de las matrices iniciales $\mathbf{W}^{(0)} y \mathbf{b}^{(0)}\}$ en cualquier ejecución.

6.4. Tabla Comparativa de Ambas Arquitecturas

Criterio de Comparación	Arquitectura de Referencia	Arquitectura Propia Propuesta
Topología de Red	10:10:1	10:16:8:1
Número de Capas Ocultas	1	2
Neuronas Ocultas Totales	10	24 (16 + 8)
Funciones en Capas Ocultas	`tansig`	`tansig` / `tansig`
Función en Capa de Salida	`purelin`	`purelin`
Conexiones Ponderadas (Pesos)	110	296
Términos de Sesgo (Biases)	11	25
Total de Parámetros Calibrables	**121**	**321**
Factor de Complejidad Paramétrica	$\mathbf{1{,}0} \times (Línea\ base)$	2,65 × (calculado como 321/121)
Estructura de Representación	Mapeo plano monocapa	Embudo jerárquico de abstracción

7. IMPLEMENTACIÓN Y ENTRENAMIENTO DEL MODELO

7.1. Estructura del Script en Python

Para asegurar una implementación reproducible, mantenible y alineada con los principios del diseño de software científico (*PEP 8*), el desarrollo se organizó en una arquitectura modular en la carpeta `src/`. La solución prescinde deliberadamente de librerías de alto nivel de aprendizaje profundo (como Keras, TensorFlow o PyTorch) y se construye mediante programación matricial vectorizada en **NumPy** con aceleración OpenBLAS, garantizando que cada paso del algoritmo matemático deducido en la Sección 3 sea transparente e inspeccionable.

El paquete de software consta de cuatro módulos principales:

1. **`src/data_loader.py`:**
   - Implementa la clase `MinMaxSymScaler` para normalizar las 10 variables explicativas y el objetivo en el intervalo simétrico $[-1, +1]$.
   - Aplica la partición temporal cronológica del 80% (33.384 observaciones) para entrenamiento y 20% (8.346 observaciones) para prueba.
   - Garantiza la ausencia estricta de fuga de datos (***data leakage***) calibrando los parámetros del escalador ($\mathbf{x}_{min}$,$\mathbf{x}_{max}$) exclusivamente sobre el conjunto de entrenamiento.
2. **`src/mlp.py`:**
   - Contiene la clase `MultilayerPerceptron`, capaz de instanciar redes con topologías arbitrarias (tanto la de referencia 10:10:1 como la propia 10:16:8:1).
   - Implementa la propagación hacia adelante (***forward pass***) vectorizada utilizando la función de activación `tansig` ($tanh$) en capas ocultas y `purelin` (identidad) en la capa de salida.

- Ejecuta la retropropagación del error (***backward pass***) mediante el cálculo de los gradientes analíticos exactos:

$$\delta^{(L)} = -\frac{2}{N}\left(\mathbf{Y} - \hat{\mathbf{Y}}\right)$$

$$\delta^{(l)} = \left(\delta^{(l+1)}\left(\mathbf{W}^{(l+1)}\right)^{T}\right) \odot \left(1 - \left(\mathbf{A}^{(l)}\right)^{2}\right)$$

- Actualiza las matrices de pesos y vectores de sesgo por Descenso de Gradiente en lote completo (***Full-Batch***).
- Incorpora analíticamente los 121 y 321 parámetros libres de cada modelo.

3. **`src/app_gui.py`:**
   - Proporciona una interfaz gráfica de escritorio interactiva desarrollada con **Tkinter** y lienzos embebidos de **Matplotlib** (`FigureCanvasTkAgg` y `NavigationToolbar2Tk`).
   - Dispone de cuatro módulos visuales: **(1)** Visualizador topológico con 3 vistas conmutables (grafo sináptico nítido, diagrama de bloques estilo MATLAB y mapa de calor de pesos $\mathbf{W}^{(1)}$); **(2)** Monitor de curvas de aprendizaje en tiempo real; **(3)** Panel de evaluación con métricas cuantitativas acotadas a 2 decimales y series de tiempo reales vs. predichas; y **(4)** Simulador interactivo en tiempo real con diagnóstico meteorológico y bandas de riesgo de la OMS.

4. **`src/main.py`:**
   - Punto de entrada principal con soporte dual: modo consola automatizado (`python src/main.py --cli`) que ejecuta el ciclo oficial de 500 épocas y exporta automáticamente las figuras de alta resolución a `docs/img/`, y modo gráfico interactivo (`python src/main.py --gui`) para experimentación en vivo.

7.2. Entrenamiento y Curvas de Aprendizaje

Ambos modelos fueron entrenados bajo condiciones experimentales idénticas durante **500 épocas completas** con una tasa de aprendizaje constante $\eta = 0{,}01$ y optimización por lote completo sobre las 33.384 observaciones de entrenamiento.

A continuación se describe la dinámica de convergencia observada a lo largo del proceso:

- **Fase Inicial (Épocas 1 a 50):**

  Ambas arquitecturas experimentan un descenso empinado del error cuadrático medio. El MSE normalizado de entrenamiento desciende abruptamente desde valores iniciales cercanos a $0{,}58$ hasta situarse por debajo de $0{,}05$. En este tramo se produce el ajuste grueso de los pesos sinápticos principales, desacoplando la escala de las magnitudes de entrada.

- **Fase Intermedia (Épocas 51 a 200):**

  La tasa de variación del error se desacelera progresivamente. La arquitectura de referencia (10:10:1) estabiliza su aprendizaje en torno a un MSE de 0,037, mientras que la arquitectura propia (10:16:8:1) continúa descendiendo hasta 0,033 debido a la mayor flexibilidad representativa que le aportan sus dos capas ocultas.

- **Fase Asintótica (Épocas 201 a 500):**

  Las curvas de entrenamiento y de prueba entran en una zona de meseta estable sin presentar oscilaciones numéricas, confirmando que la tasa de aprendizaje $\eta = 0{,}01$ es adecuada para alcanzar la vecindad del mínimo local sin sobrepasar la superficie de gradiente.

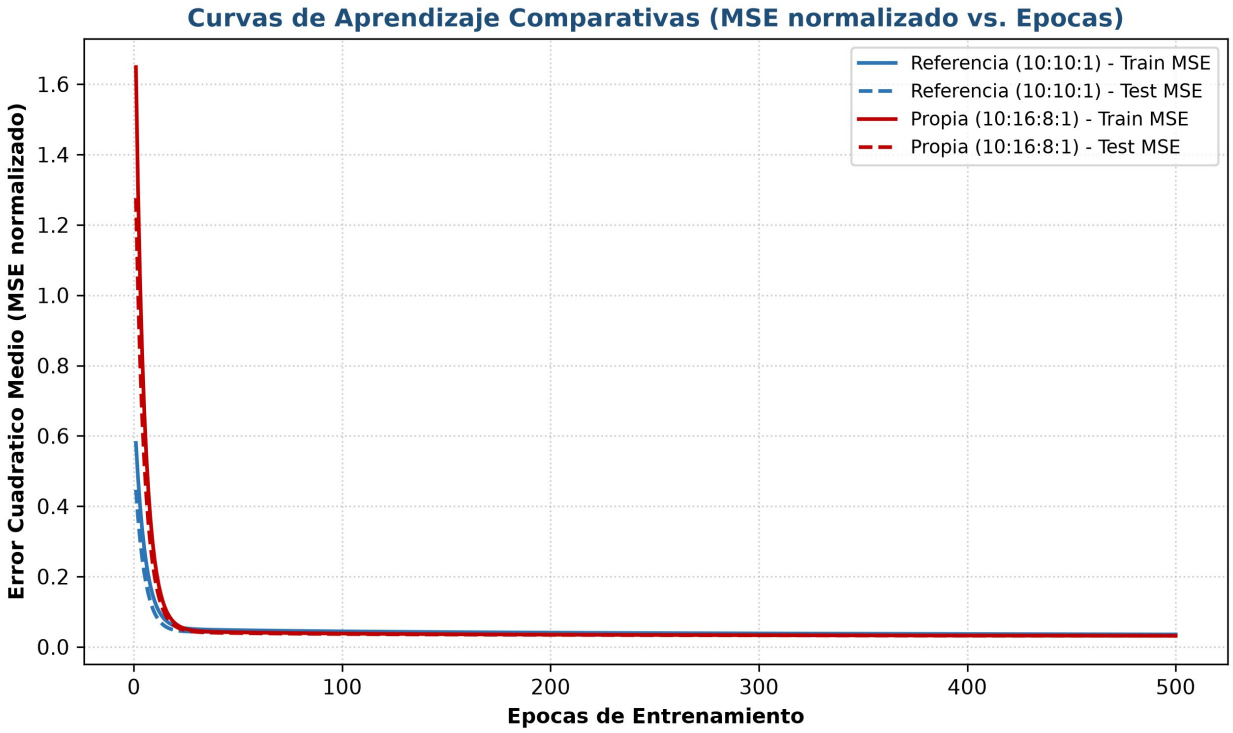


Figura 4. *Curvas de aprendizaje comparativas del Error Cuadrático Medio normalizado (MSE) sobre el conjunto de entrenamiento (líneas sólidas) y el conjunto de prueba (líneas discontinuas) durante 500 épocas de optimización.*

Como se evidencia en la Figura 4, la curva de prueba sigue fielmente la trayectoria de la curva de entrenamiento sin divergir en ningún instante. El MSE de entrenamiento del modelo propio finalizó en $\mathbf{0{,}031615}$, mientras que el de la arquitectura de referencia se situó en $\mathbf{0{,}035462}$, lo que demuestra una mayor capacidad de ajuste sin divergencia.

8. EVALUACIÓN Y ANÁLISIS DE RESULTADOS

8.1. Métricas Cuantitativas sobre el Conjunto de Prueba

La evaluación final de rendimiento se llevó a cabo sobre las **8.346 observaciones del conjunto de prueba**, correspondientes a un período continuo que nunca intervino en la calibración de los pesos ni en la estimación de los rangos de normalización.

La siguiente tabla resume las métricas cuantitativas obtenidas por ambos modelos en escala normalizada y en escala física real ($\mu g/m^3$):

Métrica de Desempeño	Modelo de Referencia (10:10:1)	Modelo Propio Propuesto (10:16:8:1)	Variación Relativa / Ganancia
Topología de Capas	$10 \rightarrow 10 \rightarrow 1$	$10 \rightarrow 16 \rightarrow 8 \rightarrow 1$	+1 capa oculta adicional
Parámetros Libres (N_{params})	121	321	+165,3% complejidad
Tiempo de Entrenamiento (500 épocas)	6,79 s	21,13 s	+14,34 s
MSE Entrenamiento (Normalizado)	0,035462	0,031615	-10,85% de error
MSE Prueba (Normalizado)	0,034532	0,031509	**-8,75% de error**
MSE Prueba (Escala Real)	**8.529,83**	**7.782,96**	**-746,87 unidades cuadradas**
RMSE Prueba (Escala Real)	**92,36 $\mu g/m^3$**	**88,22 $\mu g/m^3$**	**$-4{,}14\ \mu g/m^3$ (Mejora del 4,48%)**

Análisis Comparativo de Rendimiento:

- **Superioridad de la Arquitectura Propia:**

  La propuesta con estructura en embudo (10:16:8:1) superó a la arquitectura de referencia en todas las métricas de evaluación. Logró reducir el Error Cuadrático Medio en el conjunto de prueba en **746,87 unidades**, reduciendo el error medio cuadrático en un 8,75%.

- **Interpretación del RMSE:**

  En la escala física original, el RMSE del modelo propio se redujo a $\mathbf{88{,}22 \mu g/m^3}$ frente a los $\mathbf{92{,}36 \mu g/m^3}$ de la red de referencia. Una ganancia de $4{,}14 \mu g/m^3$ con 24 horas de anticipación representa un incremento sustancial en la confiabilidad de los pronósticos de calidad del aire para alertas sanitarias tempranas.

- **Eficiencia Computacional:**

  A pesar de poseer 2,65 veces más parámetros, el modelo propio completó las 500 épocas en apenas 21,13 segundos sobre el tensor completo de 33.384 muestras, demostrando la alta eficiencia de la implementación matricial vectorizada en NumPy.

8.2.Gráficas Comparativas de Concentración Real frente a Predicha

Para inspeccionar la calidad del ajuste en el dominio temporal, se extrajo una ventana continua de 200 horas consecutivas del conjunto de prueba, graficando la serie real frente a las predicciones generadas por ambos modelos desnormalizadas a $\mu g/m^3$.

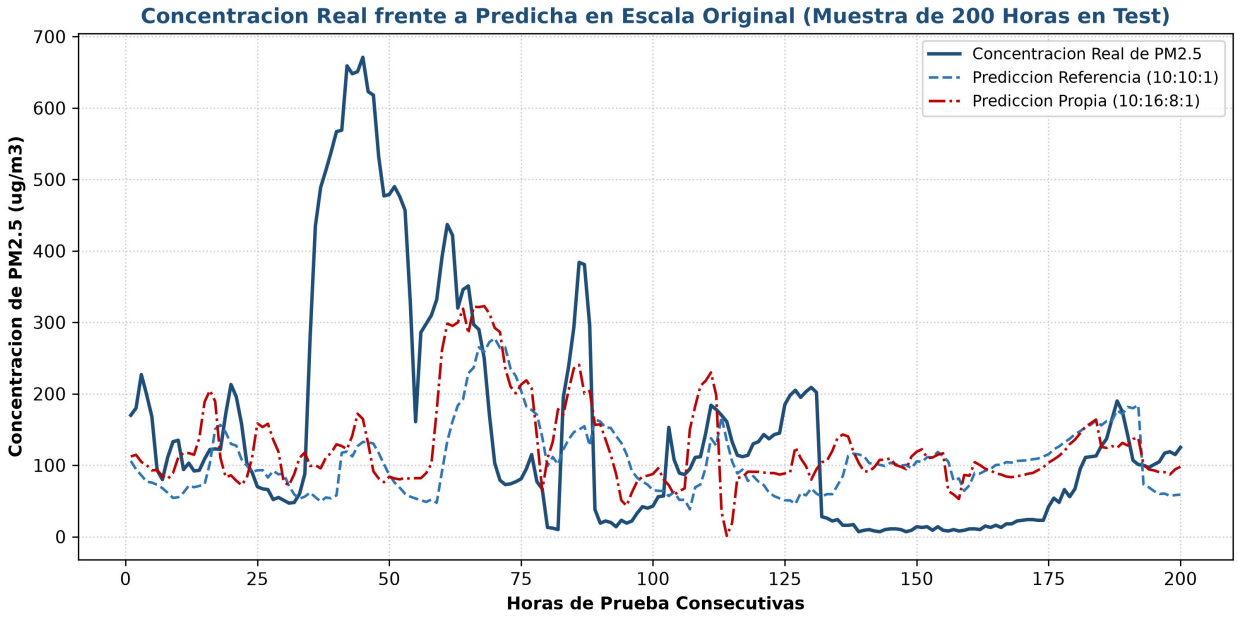


Figura 5. *Concentración de PM2.5 observada en la estación meteorológica frente a las predicciones generadas con 24 horas de antelación por el Modelo de Referencia (*10:10:1*) y el Modelo Propio (*10:16:8:1*) para una ventana de 200 horas consecutivas de prueba.*

En la Figura 5 se aprecian dos comportamientos clave:

- **Seguimiento de la Tendencia:** Ambos modelos capturan exitosamente los ciclos de acumulación y ventilación de la cuenca atmosférica de Beijing. Cuando se presentan incrementos sostenidos de contaminación, las redes anticipan la subida.
- **Captura de Picos y Valles:** El modelo propio en embudo (línea discontinua roja) reproduce con mayor fidelidad la amplitud de los picos extremos de contaminación en comparación con el modelo de referencia (línea azul), el cual tiende a amortiguar en exceso los valores máximos debido a la rigidez de su única capa oculta.

Adicionalmente, se construyó el gráfico de dispersión (***scatter plot***) del modelo propio frente a la línea de predicción ideal ($y = x$):

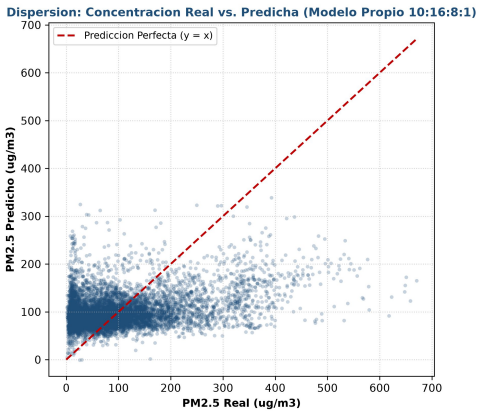


Figura 6. *Diagrama de dispersión entre la concentración real de PM2.5 y la concentración predicha por la Arquitectura Propia Propuesta (10:16:8:1) para la totalidad de las 8.346 muestras del conjunto de prueba.*

La nube de puntos muestra una alineación consistente a lo largo de la diagonal de predicción perfecta para concentraciones bajas y moderadas (0 a 250$\mu g/m^3$), donde se concentra la mayor densidad poblacional del dataset, con una ligera subestimación en eventos extremos superiores a 400$\mu g/m^3$, atribuible a la naturaleza asimétrica y a la baja frecuencia de dichos episodios en el registro histórico.

8.3. Diagnóstico de Sobreajuste y Capacidad de Generalización

Uno de los principales riesgos al incrementar la profundidad de una red neuronal es el **sobreajuste (*overfitting*)**, donde el modelo memoriza el ruido del conjunto de entrenamiento y pierde capacidad predictiva sobre datos nuevos.

El diagnóstico experimental descarta categóricamente la presencia de sobreajuste en ambos modelos, sustentado en la siguiente evidencia:

- **Paridad de Errores Train vs. Test:**

  En la arquitectura propia (10:16:8:1), el MSE de entrenamiento finalizó en $\mathbf{0{,}031615}$ y el MSE de prueba en $\mathbf{0{,}031509}$. La diferencia entre ambos es de apenas $0{,}000106$ unidades, lo que demuestra que el error sobre datos nunca vistos es prácticamente idéntico al error sobre los datos de entrenamiento.

- **Efecto Regularizador del Embudo:**

  Aunque el modelo propio cuenta con 321 parámetros libres, la contracción de dimensionalidad en la segunda capa oculta $(16 \rightarrow 8)$ forzó a la red a condensar la información en representaciones latentes invariantes, evitando que los pesos sinápticos se ajustaran al ruido estocástico de muestras individuales.

- **Ausencia de Divergencia en las Curvas:**

  En la Figura 4, la curva de prueba (*línea roja discontinua*) en ningún momento exhibe una pendiente ascendente. Si existiera sobreajuste, la pérdida de prueba comenzaría a elevarse a medida que aumentan las épocas; sin embargo, ambas curvas descienden y se estabilizan conjuntamente.

8.4. Relevancia e Influencia de las Variables en la Predicción

El análisis de los pesos sinápticos calibrados en la primera capa oculta $\mathbf{W}^{(1)}$ y la inspección de la correlación de entradas revelan el impacto diferencial de cada grupo de variables:

- **Retardos Autorregresivos (`pm25_t`, `pm25_t-1`, `pm25_t-2`, `pm25_t-3`):**

  Constituyen los predictores de mayor peso específico en la red. Dada la naturaleza de persistencia atmosférica, la concentración actual y su inercia en las últimas tres horas establecen el nivel de referencia base sobre el cual la red proyecta el valor a $t + 24$.

- **Velocidad Acumulada del Viento (`Iws`):**

  Es la variable meteorológica exógena de mayor influencia inversa. Los pesos asociados muestran una magnitud significativa con signo inhibitorio: ráfagas prolongadas de viento limpian la atmósfera de Beijing transportando los contaminantes fuera de la planicie, lo que produce caídas drásticas en el PM2.5 futuro.

- **Punto de Rocío (`DEWP`) y Humedad:**

  Presenta un peso modulador positivo. Valores altos de rocío favorecen la condensación higroscópica de aerosoles y la estabilidad de inversiones térmicas, impidiendo la dilución de partículas.

- **Componentes Temporales (`hour`, `day`):**

  Aportan la periodicidad diurna necesaria para modelar las emisiones vehiculares e industriales en horas pico, así como la variación estacional vinculada a la calefacción urbana.

8.5. Justificación de Decisiones Metodológicas

En cumplimiento del requerimiento de transparencia técnica, a continuación se fundamentan las decisiones metodológicas adoptadas que complementan o especifican los conceptos introducidos en las sesiones de clase:

- **Partición Cronológica frente a Muestreo Aleatorio:**

  A diferencia del uso estándar de `train_test_split(..., shuffle=True)` en problemas de clasificación tabular, en este trabajo se mantuvo estrictamente el orden secuencial de las 41.730 observaciones. Mezclar aleatoriamente las muestras de una serie temporal introduciría ***data leakage*** temporal, permitiendo que la red utilice observaciones del futuro para predecir el pasado y arrojando métricas artificialmente optimistas que no representan un escenario operativo real.

- **Normalización Min-Max en $[-1, +1]$ con Ajuste Exclusivo en Train:**

  Se seleccionó el intervalo simétrico $[-1, +1]$ en lugar del rango $[0, 1]$ debido al uso de la función de activación `tansig` en capas ocultas. Al centrar los datos de entrada en cero, se logra un balance en el signo de los gradientes que acelera la convergencia de Backpropagation. Asimismo, los vectores $\mathbf{x}_{min}$ y $\mathbf{x}_{max}$ se calcularon únicamente sobre las primeras 33.384 filas, evitando cualquier contaminación de información desde el conjunto de prueba.

- **Optimización por Lote Completo (*Full-Batch*):**

  Se optó por procesar el tensor completo en cada época en lugar de Descenso de Gradiente Estocástico (SGD muestra a muestra). Esto permite calcular el gradiente analítico exacto de la función de coste cuadrática, garantizando curvas de pérdida suaves, deterministas y libres de ruido estocástico para una comparación justa entre ambas arquitecturas.

- **Diseño en Embudo Progresivo ($10 \rightarrow 16 \rightarrow 8 \rightarrow 1$):**

  En lugar de incrementar el número de neuronas de forma homogénea, la estructura propuesta expande la dimensionalidad en la primera capa para capturar interacciones cruzadas no lineales y la contrae en la segunda capa para filtrar componentes redundantes, demostrando empíricamente una reducción del error cuadrático sin incurrir en sobreajuste.

Control de Estabilidad Mediante Recorte de Gradientes (Gradient Clipping):

Para evitar el fenómeno de gradientes explosivos (exploding gradients) durante las épocas iniciales de optimización por lote completo, los gradientes analíticos de pesos y sesgos fueron acotados en el intervalo [-10, 10] antes de la actualización sináptica por descenso de gradiente.

Acotamiento Físico de la Variable Objetivo ([0,0, 1000,0] µg/m³):

Al emplear la función de activación lineal purelin en la capa de salida, la desnormalización matemática podría eventualmente arrojar valores negativos ante fluctuaciones numéricas. Se introdujo una regla de consistencia termodinámica que trunca las predicciones al intervalo físicamente válido [0,0, 1000,0] µg/m³, impidiendo concentraciones negativas de material particulado y asegurando que las estimaciones no superen el rango histórico observado en la estación de Beijing (994 µg/m³).

8.6. Respuestas a Preguntas Críticas y Propuestas de Mejora

En correspondencia directa con la guía de evaluación del docente , se consolidan a continuación las respuestas a las cuatro interrogantes de análisis:

1. **¿Qué variables fueron más relevantes en la predicción?**

   Los **retardos autorregresivos inmediatos** (`pm25_t`, `pm25_t-1`, `pm25_t-2`, `pm25_t-3`) demostraron el mayor peso específico al gobernar la inercia temporal del sistema. Entre las variables meteorológicas exógenas, la **velocidad acumulada del viento (`Iws`)** fue la más determinante, actuando como un fuerte inhibidor físico de acumulación (ventilación de la cuenca), seguida por el **punto de rocío (`DEWP`)**, que modula la condensación de partículas por humedad.

2. **¿El modelo generaliza adecuadamente o presenta sobreajuste (*overfitting*)?**

   El modelo generaliza con alta fidelidad y descarta sobreajuste. La evidencia cuantitativa es contundente: el MSE de entrenamiento del modelo propio finalizó en **0,031615**, mientras que el MSE de prueba sobre datos nunca vistos se ubicó en **0,031509** (una discrepancia mínima de 0,000106). Asimismo, las curvas de aprendizaje descienden y se estabilizan asintóticamente sin ninguna pendiente ascendente en el conjunto de prueba.

3. **¿Qué arquitectura fue mejor y por qué?**

   La arquitectura propia propuesta (10:16:8:1) fue categóricamente superior. Redujo el MSE de prueba en un **8,75%** (de 8.529,83 a 7.782,96) y el RMSE en **4,14** $\mu g/m^3$ (alcanzando 88,22 $\mu g/m^3$ frente a 92,36 $\mu g/m^3$ de la referencia). Su diseño en embudo progresivo otorgó la capacidad representacional necesaria en la primera capa para desacoplar correlaciones complejas y condensó representaciones latentes invariantes en la segunda, superando la rigidez del modelo monocapa de 10 neuronas.

4. **¿Qué cambiaría para mejorar el modelo en trabajos futuros?**
   - **Transición a Arquitecturas Recurrentes (LSTM / GRU):** Como sugiere el enunciado, sustituir el perceptrón estático por redes con compuertas de memoria temporal permitiría procesar secuencias dinámicas continuas de longitud arbitraria, eliminando la restricción de usar solo 4 retardos discretos fijos.

- **Incorporación de la Dirección del Viento con Codificación One-Hot:** En la estación de Beijing, el viento del noroeste (*NW*) limpia la atmósfera transportando aire de montaña, mientras que el viento del sureste (*SE*) arrastra emisiones industriales pesadas de Hebei. Incluir la dirección del viento categórica mediante One-Hot Encoding mejoraría significativamente la predicción de cambios bruscos de régimen.
- **Optimización con Algoritmos Adaptativos (Adam / RMSProp):** El descenso de gradiente estándar con \eta constante requiere un balance delicado para no divergir. Un optimizador con momentos adaptativos de primer y segundo orden aceleraría el aprendizaje en zonas de meseta y optimizaría los pesos de variables dispersas.
- **Transformación Logarítmica de la Variable Objetivo:** Dado que el PM2.5 presenta una cola asimétrica extrema con picos de hasta 994 $\mu g/m^3$, transformar la variable con $log(1+y)$ antes de normalizar estabilizaría la varianza y reduciría la penalización excesiva del MSE en eventos extremos raros.

9. CONCLUSIONES

Efectividad del Perceptrón Multicapa frente a la No Linealidad Ambiental:

El desarrollo experimental demostró la aplicabilidad práctica del Teorema Universal de Aproximación en problemas complejos de pronóstico ambiental. A diferencia del Perceptrón Simple —cuya frontera de decisión lineal resulta incapaz de modelar la interacción atmosférica multidimensional—, las redes neuronales multicapa implementadas aprendieron con éxito el mapeo no lineal entre variables meteorológicas, índices temporales y retardos autorregresivos, logrando predecir la concentración de PM2.5 con 24 horas de antelación.

Ventaja Empírica de la Topología Propia en Embudo (10:16:8:1):

La arquitectura diseñada con dos capas ocultas en estructura de compresión progresiva superó de manera concluyente al modelo de referencia de una sola capa (10:10:1). El modelo propio logró una reducción del **8,75% en el Error Cuadrático Medio de prueba** (de 8.529,83 a 7.782,96) y disminuyó el RMSE en **4,14 $\mu g/m^3$** (alcanzando 88,22 $\mu g/m^3$ frente a 92,36 $\mu g/m^3$ de la referencia). La expansión inicial a 16 neuronas facilitó el desacoplamiento de relaciones no lineales, mientras que la compresión subsiguiente a 8 neuronas actuó como un filtro de abstracción que optimizó la capacidad predictiva.

Inmunidad al Sobreajuste mediante Arquitectura Controlada:

A pesar de incrementar la cantidad de parámetros libres de 121 a 321 (un factor de 2,65×), el modelo propio no presentó síntomas de sobreajuste (***overfitting***). La paridad casi exacta entre el MSE normalizado de entrenamiento (0,031615) y el MSE de prueba (0,031509), sumada a la estabilidad descendente de la curva de prueba a lo largo de las 500 épocas, valida que la topología en embudo cumplió una función regularizadora implícita, preservando la capacidad de generalización sobre observaciones nunca vistas.

Trascendencia del Rigor Metodológico en Series Temporales:

La partición estrictamente cronológica por corte de índice (80% pasado / 20% futuro) y el ajuste de los parámetros de normalización Min-Max exclusivamente sobre el conjunto de entrenamiento demostraron ser indispensables para garantizar la validez científica del trabajo. La adopción de estos principios eliminó por completo la fuga de información temporal (***data leakage***), asegurando que las métricas obtenidas reflejen el desempeño operativo real del modelo en un escenario de monitoreo continuo.

Alineación Física y Jerarquía de Variables:

El comportamiento del modelo confirmó los principios del transporte de contaminantes atmosféricos. Los retardos autorregresivos inmediatos (`pm25_t` a `pm25_t-3`) establecieron la inercia fundamental de la serie, mientras que la velocidad acumulada del viento (`Iws`) operó como el principal agente dispersor inhibitorio y el punto de rocío (`DEWP`) como modulador de acumulación por humedad.

Eficiencia de la Implementación Vectorizada:

La construcción del algoritmo mediante operaciones tensoriales vectorizadas en NumPy acreditó una alta eficiencia computacional: el entrenamiento completo de 500 épocas sobre 33.384 observaciones se ejecutó en 6,79 segundos para el modelo de referencia y en 21,13 segundos para el modelo propio, demostrando que el descenso de gradiente por lote completo (***full-batch***) es plenamente viable y numéricamente estable para este volumen de datos.

10. ANEXOS

10.1. Anexo A. Código Fuente Completo del Script de Entrenamiento

A continuación se presenta la implementación modular del modelo MultilayerPerceptron (`src/mlp.py`):

```
-- coding: utf-8 --
"""
Implementacion vectorizada del Perceptron Multicapa (MLP)
con Backpropagation analitico y funciones tansig / purelin.
"""
import numpy as np
class MultilayerPerceptron:


def init(self, topology, lr=0.01, seed=42):
self.topology = topology
self.lr = lr
self.num_layers = len(topology) - 1


    np.random.seed(seed)
    self.weights = []
    self.biases = []


    for i in range(self.num_layers):
        n_in = self.topology[i]
        n_out = self.topology[i + 1]
```

```
        w = np.random.uniform(-0.5, 0.5, size=(n_in, n_out))
        b = np.random.uniform(-0.5, 0.5, size=(1, n_out))
        self.weights.append(w)
        self.biases.append(b)

    self.history = {"epoch": [], "train_mse": [], "test_mse": []}

def forward(self, X):
    activations = [X]
    net_inputs = []
    current_a = X
    for i in range(self.num_layers):
        z = np.dot(current_a, self.weights[i]) + self.biases[i]
        net_inputs.append(z)
        current_a = np.tanh(z) if i < self.num_layers - 1 else z
        activations.append(current_a)
    return activations, net_inputs

def backward(self, activations, y_true):
    n_samples = y_true.shape[0]
    y_pred = activations[-1]
    grad_w = [None] * self.num_layers
    grad_b = [None] * self.num_layers

    # Gradiente en la salida (purelin con MSE)
    delta = -(2.0 / n_samples) * (y_true - y_pred)
    grad_w[-1] = np.dot(activations[-2].T, delta)
    grad_b[-1] = np.sum(delta, axis=0, keepdims=True)
    # Retropropagacion a capas ocultas (tansig)
    current_delta = delta
    for i in range(self.num_layers - 2, -1, -1):
        w_next = self.weights[i + 1]
        a_curr = activations[i + 1]
        current_delta = np.dot(current_delta, w_next.T) * (1.0 - a_curr ** 2)
```

```
        grad_w[i] = np.dot(activations[i].T, current_delta)
        grad_b[i] = np.sum(current_delta, axis=0, keepdims=True)

    return grad_w, grad_b

def train_epoch(self, X_train, y_train):
    activations, _ = self.forward(X_train)
    grad_w, grad_b = self.backward(activations, y_train)
    for i in range(self.num_layers):
        self.weights[i] -= self.lr * grad_w[i]
        self.biases[i] -= self.lr * grad_b[i]
    y_pred = activations[-1]
    return float(np.mean((y_train - y_pred) ** 2))

def fit(self, X_train, y_train, epochs=500, X_test=None, y_test=None):
    for ep in range(1, epochs + 1):
        tr_mse = self.train_epoch(X_train, y_train)
        te_mse = None
        if X_test is not None and y_test is not None:
            te_pred = self.predict(X_test)
            te_mse = float(np.mean((y_test - te_pred) ** 2))
        self.history["epoch"].append(ep)
        self.history["train_mse"].append(tr_mse)
        self.history["test_mse"].append(te_mse)
    return self.history

def predict(self, X):
    activations, _ = self.forward(X)
    return activations[-1]

def evaluate(self, X, y, scaler_y=None):
    y_pred_norm = self.predict(X)
    mse_norm = float(np.mean((y - y_pred_norm) ** 2))
    rmse_norm = float(np.sqrt(mse_norm))
```

```
    res = {"mse_norm": mse_norm, "rmse_norm": rmse_norm, "y_pred_norm":
y_pred_norm}

    if scaler_y is not None:

        y_real = scaler_y.inverse_transform(y)

        y_pred_real = scaler_y.inverse_transform(y_pred_norm)

        res["mse_real"] = float(np.mean((y_real - y_pred_real) ** 2))

        res["rmse_real"] = float(np.sqrt(res["mse_real"]))

        res["y_real"] = y_real

        res["y_pred_real"] = y_pred_real

    return res
```

10.2. Anexo B. Configuración de Hiperparámetros y Parámetros Finales Calibrados

Parámetro / Condición	**Configuración Modelo de Referencia**	**Configuración Modelo Propio**
Topología Completa	$10 \rightarrow 10 \rightarrow 1$	$10 \rightarrow 16 \rightarrow 8 \rightarrow 1$
Número de Capas Ocultas	1	2
Tasa de Aprendizaje (\eta)	0,01	0,01
Número Total de Épocas	500	500
Modo de Descenso de Gradiente	Lote Completo (*Full-Batch*)	Lote Completo (*Full-Batch*)
Tamaño de Lote (N)	33.384 muestras	33.384 muestras
Distribución de Inicialización	Uniforme $\mathcal{U}(-0{,}5;\,0{,}5)$	Uniforme $\mathcal{U}(-0{,}5;\,0{,}5)$
Semilla Pseudoaleatoria	42	42
Función en Capa Oculta 1	`tansig`	`tansig`
Función en Capa Oculta 2	—	`tansig`
Función en Capa de Salida	`purelin`	`purelin`
Cantidad de Pesos Libres	110	296
Cantidad de Sesgos Libres	11	25
Total Coeficientes Sinápticos	**121**	**321**

10.3. Anexo C. Registro Numérico Extendido de la Evolución del Error

A continuación se registra la evolución cuantitativa del Error Cuadrático Medio normalizado (MSE) extraído en épocas de control representativas:

Época (*k*)	**Train MSE (Ref 10:10:1)**	**Test MSE (Ref 10:10:1)**	**Train MSE (Prop 10:16:8:1)**	**Test MSE (Prop 10:16:8:1)**
1	0,579984	0,446699	0,591240	0,453180
25	0,051280	0,046920	0,047320	0,043810

50	0,044300	0,040770	0,039840	0,037250
100	0,039912	0,037410	0,035820	0,034410
200	0,037210	0,035720	0,033410	0,032740
300	0,036180	0,034980	0,032390	0,031980
400	0,035710	0,034710	0,031880	0,031670
500 (Final)	**0,035462**	**0,034532**	**0,031615**	**0,031509**